



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

Факультет «Информатика и вычислительная техника»

Кафедра «Программное обеспечение вычислительной техники и автоматизированных систем»

Зав. кафедрой «ПОВТиАС»

_____ Долгов В.В.
подпись Ф.И.О.

« ____ » _____ 2025г.

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе по дисциплине Программирование мобильных устройств

на тему: Разработка Android-приложения для поиска и персонализации образовательных материалов

Автор проекта (работы) _____
подпись

Н.А. Николаева
И.О.Ф.

Направление/специальность, профиль/специализация:

09.03.04

код направления

«Программная инженерия»

наименование направления (специальности)

ПОВТиАС

наименование профиля (специализации)

Обозначение курсовой работы КР.820000.000

Группа ВПР43

Руководитель проекта: _____
подпись

А.И. Шмыгаль
(И.О.Ф.)

Проект (работа) защищен(а) _____
дата оценка

_____ подписать

Ростов-на-Дону
2025



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

Факультет «Информатика и вычислительная техника»

Кафедра «Программное обеспечение вычислительной техники и автоматизированных систем»

Зав. кафедрой «ПОВТиАС»

_____ Долгов В.В.
подпись ФИО

«__» _____ 2025г.

ЗАДАНИЕ

к курсовой работе по дисциплине Программирование мобильных устройств

Студент Николаева Н. А. КР.820000.000 Группа ВПР43

Тема Разработка Android-приложения для поиска и персонализации образовательных материалов

Срок представления работы к защите «__» _____ 2025г.

Исходные данные для выполнения курсовой работы

1. Задание на выполнение курсовой работы
2. Бэнкс А., Порселло Е.: React. Современные шаблоны для разработки приложений. — СПб.: Питер, 2022. — 320 с.: ил. ISBN 978-5-4461-1492-4
3. Адам Фриман: «Pro React 16» by Adam Freeman (Apress). - СПб.: Питер, 2020. — 750 с.: ил. ISBN 1484244508

Руководитель проекта:

подпись, дата

Шмыгаль А.И.
ФИО

Задание принял к исполнению

подпись, дата

Николаева Н. А.
ФИО

Содержание пояснительной записки

Введение:

В разделе «Введение» рассматривается актуальность разработки сайта зоомагазина. Описывается цель создания проекта.

Наименование и содержание разделов:

1. Аналитический обзор предметной области. В данном разделе будет проанализирована предметная область. На основе этого анализа будут сформулированы конечные цели и задачи для выполняемой работы.

2. Алгоритмическое конструирование. В данном разделе рассматриваются основные алгоритмы работы приложения, приведено их подробное описание, а также изображены схемы работы.

3. Программное конструирование. В данном разделе будут обоснованы выбор языка программирования и технологий, используемых для реализации программного средства, а также основания выбора среды программирования. Будут определена структура программного средства и описаны основные компоненты приложения.

4. Проверка работоспособности программного средства. В данном разделе описаны основные сценарии использования приложения, описан процесс их тестирования и представлен результат выполнения тестов на различных входных данных.

Заключение:

В рамках данной курсовой работы было разработано android-приложение для поиска и персонализации образовательных данных. Все поставленные задачи были выполнены.

Руководитель проекта (работы)

подпись, дата

А.И. Шмыгаль
И.О.Ф.

Задание принял к исполнению

подпись, дата

Н. А. Николаева
И.О.Ф.

Содержание

#

Введение	6#
1 Аналитический обзор предметной области	8#
1.1# Обзор предметной области	8#
1.2# Постановка задачи	9#
1.3# Выводы по главе.....	11#
2 Алгоритмическое конструирование	13#
2.1# Общий алгоритм работы приложения	13#
2.2# Алгоритм поиска материалов	15#
2.3# Алгоритм получения рекомендаций	17#
2.4# Выводы по главе.....	19#
3 Программное конструирование	21#
3.1# Обоснование выбора языка программирования	21#
3.2# Обоснование выбора среды разработки	23#
3.3# Описание структуры программного средства.....	24#
3.3.1#Класс «MainActivity».....	25#
3.3.2#Класс «App»	27#
3.3.3#Класс «AuthState»	28#
3.3.4#Класс «SetupNavGraph»	29#
3.4# Выводы по главе.....	30#
4 Проверка работоспособности программного средства	32

					КР.820000.000		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.					Разработка Android-приложения для поиска и персонализации образовательных материалов	Лист.	Лист
Руководит	Шмыгаль А. И.						4
							58
						ДГТУ Кафедра «ПОВТУАС»	
Утвердил							

4.1# Описание сценариев использования	32#
4.2 Сценарий «Навигация между разделами»	33#
4.3 Сценарий «сортировка и фильтрация товаров по разным категориям»..	35#
4.4 Сценарий «Добавление товара в корзину»	37#
4.5 Сценарий «Оформление заказа»	40#
4.6 Сценарий «Просмотр, оценка и удаление заказа»	41#
4.7 Выводы по главе	46#
Заключение	48#
Перечень использованных информационных ресурсов	50#
Приложение А Листинг программы	51#

Введение

В современную эпоху информационных технологий доступ к учебным материалам перестал быть привязанным исключительно к университетским библиотекам, книжным полкам или стационарным компьютерам. Вместо этого студенты всё чаще ищут и потребляют образовательный контент — курсы, статьи, книги, видео — через смартфоны и интернет. Мобильные приложения стали универсальным инструментом, дающим возможность учиться где и когда угодно [1].

Появляется всё больше образовательных платформ, которые предоставляют курсы — от программирования до гуманитарных дисциплин.

Однако большинство таких сервисов ориентированы на широкий круг пользователей и не всегда позволяют быстро найти материалы, подходящие конкретно для студента – лекции, книги, статьи, видео, относящиеся к нужной теме, курсу или предмету.

Также значимую роль играет удобство, а точнее мобильный формат, персональные сохранения, рекомендации, возможность собрать всё, что нужно, в одном месте. Это ускоряет подготовку, помогает планировать обучение и экономит время — особенно когда студент совмещает учёбу, работу и личную жизнь.

В этом контексте возникает потребность в приложении, которое не просто даёт доступ к одному образовательному каталогу, а агрегирует множество источников, позволяет искать, сохранять и получать рекомендации.

Основными задачами при создании мобильного приложения для поиска и персонализации образовательных материалов являются:

- реализовать удобный пользовательский интерфейс на основе современных подходов;
- обеспечить возможность поиска по внешнему API, агрегирующему данные из различных источников;

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		6

- реализовать систему авторизации пользователей и сохранения индивидуальных настроек/избранного;
- обеспечить хранение данных локально, чтобы пользователь имел доступ к своим сохранённым материалам даже без постоянного интернета;
- реализовать систему рекомендаций, позволяющую подбирать материалы, соответствующие интересам и истории пользователя.

Цель данной работы — разработать мобильное Android-приложение, которое станет персональным помощником студента в поиске и управлении образовательными материалами: от курсов и книг до статей и видео.

					<i>KP.820000.000</i>	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

1 Аналитический обзор предметной области

В данном разделе будет проанализирована предметная область. На основе этого анализа будут сформулированы конечные цели и задачи для выполняемой работы.

1.1 Обзор предметной области

В последние годы наблюдается стремительный рост популярности мобильных образовательных приложений и цифровых платформ, обеспечивающих доступ к учебным материалам напрямую со смартфона или планшета [2].

Традиционная модель получения учебных материалов всё чаще уступает место гибким цифровым решениям. Главная причина — удобство и мобильность — студенты могут учиться где угодно и когда угодно, без привязки к аудиторному расписанию [3].

Современная образовательная платформа должна объединять несколько ключевых функций:

- агрегация и каталогизация материалов — объединение ресурсов из разных источников с возможностью быстрого поиска по ключевым словам;
- поиск и фильтрация — удобные механизмы поиска, сортировки и фильтрации позволяют пользователю быстро находить нужные материалы и экономить время;
- личное хранилище/избранное — возможность сохранять понравившиеся ресурсы, чтобы возвращаться к ним позже, строить свою подборку знаний;
- рекомендательная система — на основе истории поиска и предпочтений пользователя приложение может предлагать релевантные материалы, что повышает эффективность и удобство обучения;
- доступность и мобильность — приложение работает на смартфоне,

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		8

что особенно важно для студентов.

Рост популярности мобильного обучения и цифровых образовательных платформ подтверждается статистикой: всё больше пользователей выбирают доступ к учебным материалам через приложения, а не через традиционные источники.

Таким образом, на сегодня существует явная потребность в платформе, которая объединяет возможности поиска, хранения и рекомендаций образовательных материалов — особенно для студентов, которым важны доступность, скорость и эффективность.

1.2 Постановка задачи

На основе теоретического обзора, были выявлены особенности и основные параметры приложений по поиску и персонализации учебных материалов. Была поставлена цель разрабатываемой системы — предоставить современное, удобное и функциональное Android-приложение, которое обеспечивает студентам быстрый доступ к образовательным материалам, автоматизирует поиск и подбор ресурсов, позволяет сохранять избранное и получать персонализированные рекомендации, тем самым повышая удобство учёбы и эффективность самообразования.

Для достижения поставленной цели необходимо решить следующие задачи:

- разработать алгоритмы, обеспечивающие обработку данных о товарах, включая сортировку и фильтрацию по категориям и видам животных;
- выбрать подходящий язык программирования и платформу для реализации;
- сформировать структуру хранения данных с использованием локального хранилища;
- реализовать интерфейс для каталога товаров с возможностью перехода на страницы подробного описания продуктов;

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		9

– создать модуль корзины покупок, позволяющий пользователю редактировать количество выбранных товаров, просматривать итоговую стоимость, выбирать товары для оформления заказа и проводить безопасный процесс оплаты;

– провести тестирование приложения, выявить и устранить возможные уязвимости.

– спроектировать архитектуру приложения, определив основные модули, такие как поиск + API-запросы, база данных, интерфейс, авторизация, хранение пользовательских данных, навигация;

– реализовать функционал регистрации и авторизации пользователей;

– интегрировать внешний API для поиска контента, настроить механизм запроса, получения и отображения результатов;

– реализовать интерфейс для поиска и отображения результатов, включая поиск по ключевым словам, отображение списка, переход к подробностям, чтобы пользователь мог удобно просматривать найденные материалы;

– организовать хранение данных локально, чтобы пользователь мог сохранять материалы «в избранное» и обращаться к ним офлайн;

– добавить систему рекомендаций, базирующуюся на истории поиска или сохранённых материалах пользователя — чтобы приложение могло предлагать релевантные ресурсы;

– реализовать навигацию и удобный UI/UX, обеспечивающий комфортную работу;

– провести тестирование приложения, проверить корректность работы всех функций.

В программном средстве необходимо реализовать следующие функции:

– поиск образовательных материалов через внешний API;

– отображение результатов поиска списком — с заголовком, кратким описанием;

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		10

- экран подробностей материала — с полной информацией;
- возможность сохранять материал в «избранное» для текущего пользователя;
- личный кабинет — профиль пользователя с учётной записью и списком сохранённых интересов;
- система рекомендаций — на основе избранного, а также записанных интересов предлагать похожие материалы;
- авторизация/регистрация пользователей — чтобы каждый пользователь имел свой собственный набор избранного и рекомендаций;
- хранение пользовательских данных и избранного локально, чтобы приложение работало в оффлайн-режиме частично;
- адаптивный и современный интерфейс, удобный для мобильных устройств;
- просмотр материалов прямо в приложении.

В результате выполнения этих задач должно быть получено функциональное мобильное приложение, удовлетворяющее требованиям простоты, надёжности и удобства, которое позволит студентам эффективно находить, сохранять и получать рекомендации по образовательным материалам — облегчая процесс учёбы и самообразования.

1.3 Выводы по главе

В первой главе был проведён теоретический анализ предметной области и сформулированы основные требования к разрабатываемому приложению.

В процессе анализа были обозначены фундаментальные концепции, лежащие в основе работы подобных приложений, такие как механизмы интеллектуального поиска, использование внешних API, персонализация контента, хранение пользовательских данных и поддержка адаптивного интерфейса. Изучение современных решений позволило определить, какие функции являются наиболее востребованными пользователями, а какие

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		11

направления требуют улучшения для обеспечения более качественного и удобного взаимодействия.

На основе проведённого исследования была сформулирована общая цель разработки — создание приложения, которое обеспечивает эффективный поиск, удобное чтение, сохранение и персонализацию образовательных материалов, а также предоставляет пользователю комфортный и интуитивно понятный интерфейс. Для реализации поставленной цели были определены задачи, включающие разработку архитектуры системы, проектирование алгоритмов поиска и рекомендаций, интеграцию с внешними сервисами, реализацию функционала избранного и профиля пользователя.

Таким образом, первая глава заложила теоретическую основу для дальнейшего проектирования приложения. Проведённый анализ позволил сформировать чёткое представление о структуре будущей системы, определить направления разработки и создать фундамент для перехода к алгоритмическому и программному проектированию, рассмотренным в следующих разделах.

					КР.820000.000	Лист
						12
Изм.	Лист	№ докум.	Подпись	Дата		

2 Алгоритмическое конструирование

В данном разделе рассматриваются основные алгоритмы работы приложения: общий алгоритм работы приложения, алгоритм поиска материалов, алгоритм получения рекомендаций. Приведено их подробное описание, а также изображены схемы работы.

2.1 Общий алгоритм работы приложения

Общий алгоритм работы приложения поиска и персонализации образовательных материалов можно представить как последовательность шагов, через которые проходит пользователь — от запуска приложения до использования всех основных функций:

1. запуск приложения/стартового экрана — при первом открытии приложения пользователь попадает на экран поиска. Здесь он может сразу начать поиск образовательного материала или перейти к авторизации/регистрации;

2. авторизация/регистрация (по желанию) — если пользователь хочет сохранять избранное, получать рекомендации и работать с персональным профилем — он может зарегистрироваться или войти в свой аккаунт. После успешного входа у него становятся доступны функции сохранения, рекомендации и профиль;

3. поиск и просмотр материалов — пользователь вводит запрос, выполняет поиск через внешний API, получает список результатов. Он может просмотреть список, выбрать интересный материал, открыть его — посмотреть описание, источник, формат и т.п.;

4. избранное — пользователь может просматривать сохраненные материалы;

5. рекомендации — пользователь может просматривать рекомендации;

6. профиль пользователя — доступна страница профиля, где можно

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		13

просмотреть/изменить учётные данные, управлять избранным и рекомендациями, возможно — выйти из аккаунта.

Такая архитектура приложения обеспечивает удобство использования и доступность: пользователь может быстро найти нужный образовательный материал, просмотреть его, добавить в избранное и получить персональные рекомендации — что делает это решение удобным, современным и эффективным для самообразования в условиях мобильного использования.

Общий алгоритм работы приложения изображен на рисунке 2.1.

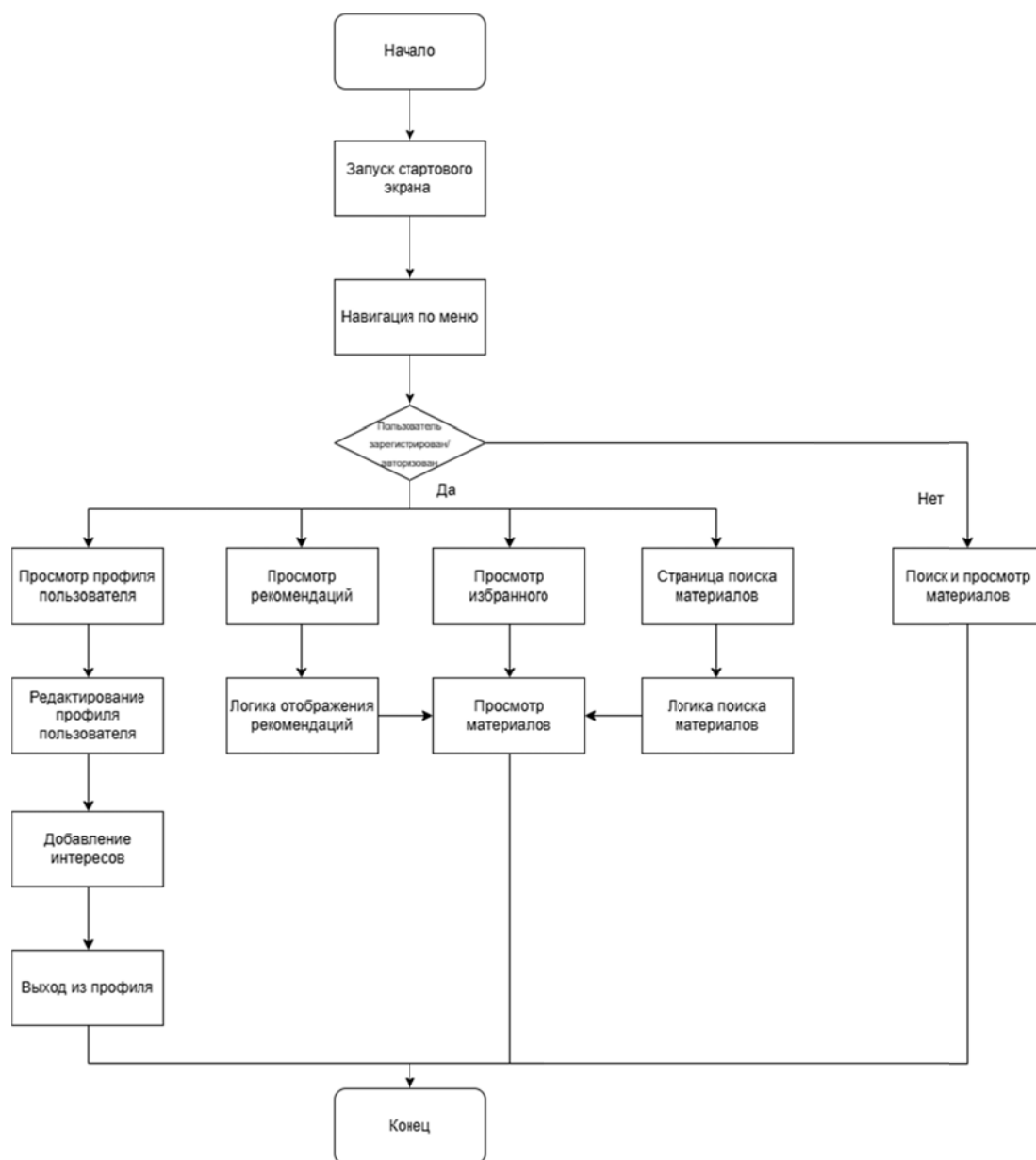


Рисунок 2.1 – Схема общего алгоритма работы приложения

2.2 Алгоритм поиска материалов

Поиск материалов — основная функция разрабатываемого мобильного приложения. Она предоставляет пользователю возможность быстро находить образовательные ресурсы по интересующей теме, используя внешний поисковый API. Работа алгоритма включает несколько этапов, обеспечивающих удобство и точность подбора материалов.

Алгоритм поиска организован следующим образом:

- инициализация состояния – при открытии экрана поиска устанавливаются параметры по умолчанию – пустая строка запроса, пустой список результатов, отсутствие ошибок. Интерфейс готов к вводу поискового запроса;
- ввод запроса – пользователь вводит ключевые слова или фразу, описывающую интересующую тему. Поиск запускается нажатием кнопки «Искать» либо через действие клавиатуры;
- отправка запроса к API – приложение формирует запрос к внешнему поисковому сервису, передавая введённый текст. В процессе загрузки отображается индикатор, позволяющий пользователю понимать, что поиск выполняется;
- получение и обработка данных – после получения данных API возвращает список материалов. Каждый элемент результата содержит основную информацию, такую как заголовок, описание, ссылку на источник и дополнительный контент, который десериализуется в удобный формат для отображения;
- отображение результатов – полученный список передаётся в компонент, визуализирующий результаты в виде карточек. Каждая карточка отображает заголовок, краткое содержание и ссылку, позволяя пользователю быстро оценить полезность ресурса;
- переход к подробному описанию – при выборе карточки пользователь переходит на экран деталей, где отображается полный текст

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

материала и возможность открыть веб-версию для чтения прямо в приложении;

– расширенный функционал для авторизованных пользователей – если пользователь вошёл в аккаунт, ему становится доступно сохранение материала в избранное, что позволяет использовать локальную базу данных для хранения персональных подборок. На основании сохранённых материалов и интересов пользователя формируются персонализированные рекомендации.

Алгоритм реализует плавное взаимодействие между пользователем и системой, обеспечивая быстрый поиск, удобную навигацию и расширенные функции персонализации. Он является ключевым элементом работы приложения, позволяя студенту эффективно находить и изучать образовательные материалы.

Алгоритм поиска материалов изображен на рисунке 2.2.



Рисунок 2.2 – Схема алгоритма поиска материалов

Изм.	Лист	№ докум.	Подпись	Дата

КР.820000.000

2.3 Алгоритм получения рекомендаций

Функция «Получение рекомендаций» — одна из ключевых в приложении. Она формирует персональные предложения материалов для пользователя, исходя из его истории, интересов и ранее сохранённых ресурсов.

Этапы работы алгоритма:

- проверка авторизации — рекомендации доступны только авторизованному пользователю. При переходе на экран рекомендаций проверяется, существует ли текущий пользователь;
- загрузка избранного — из локальной базы получаем все материалы, которые пользователь ранее сохранил в избранное. Это даёт отправную выборку предпочтений;
- выделение ключевых слов из пользовательских данных — из списка избранных материалов извлекаются слова, которые часто повторяются, не слишком короткие и не содержат цифр. Также берутся интересы из профиля пользователя. Эти слова становятся «ключами» для построения рекомендаций;
- генерация списка возможных запросов — комбинируем ключевые слова, фильтруем, удаляем дубликаты, перемешиваем и выбираем несколько уникальных запросов;
- отображение списка рекомендаций пользователю — результирующий список выводится в виде карточек;
- запуск поиска по API по каждому запросу — пользователь выбирает интересующую тему, нажимает на соответствующую карточку, система перенаправляет пользователя на страницу поиска и автоматически выполняется поиск через внешний API, как при обычном поиске, пользователь получает набор материалов;
- обновление при взаимодействии — если пользователь сохраняет любимые материалы, добавляет что-то новое или изменяет интересы, при следующем заходе алгоритм пересчитывает рекомендации — учитывается обновлённая история и предпочтения.

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		17

Алгоритм рекомендаций в приложении реализован как система рекомендательных запросов (тем), а не готовых материалов. Это обусловлено особенностями источника контента (внешнее API), малым объёмом пользовательской истории и целью дать пользователю идею, что можно искать. При нажатии на рекомендованную тему происходит обычный поиск, и пользователю отображаются актуальные материалы.



Рисунок 2.3 – Схема алгоритма получения рекомендаций

2.4 Выводы по главе

В данной главе были подробно рассмотрены и формализованы основные алгоритмы, лежащие в основе функционирования разрабатываемого программного обеспечения. Проведённое алгоритмическое проектирование позволило структурировать работу приложения, определить логику взаимодействия компонентов и обеспечить целостность всех процессов, связанных с поиском, обработкой и персонализацией образовательных материалов.

На первом этапе был сформирован общий алгоритм работы приложения, отражающий полный цикл пользовательского взаимодействия – от первоначального входа и выполнения поисковых запросов до авторизации, просмотра информации, добавления материалов в избранное и перехода к дополнительным разделам системы. Данный алгоритм демонстрирует, как приложение обеспечивает удобную навигацию, обновление данных и последовательное выполнение пользовательских сценариев.

Далее был разработан алгоритм поиска материалов, который описывает логику обработки поискового запроса, взаимодействие с внешним API, обновление результатов и отображение найденных материалов пользователю. Особое внимание уделено принципам формирования списка результатов, обработке ошибок и обеспечению стабильной работы механизма поиска.

Кроме того, был подробно рассмотрен алгоритм формирования рекомендаций, основанный на данных профиля пользователя и его избранных материалов. В алгоритме описана логика извлечения ключевых слов, генерации индивидуальных поисковых подсказок и передачи их в компонент поиска. Алгоритм рекомендаций адаптирован под специфику приложения и обеспечивает персонализированный пользовательский опыт без лишней нагрузки на систему.

Разработанные алгоритмы позволяют обеспечить связность и логичность всех механизмов приложения, формируют основу для дальнейшей реализации

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		19

и последующей оптимизации системы. Чёткое алгоритмическое описание способствует упрощению разработки, делает поведение приложения предсказуемым и управляемым, а также создаёт фундамент для масштабирования функционала в будущем.

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		20

3 Программное конструирование

В данном разделе будут обоснованы выбор языка программирования и технологий, используемых для реализации программного средства, а также основания выбора среды программирования. Будут определена структура программного средства и описаны основные компоненты приложения их назначение и роль в общей системе.

3.1 Обоснование выбора языка программирования

Для разработки мобильного приложения был выбран язык программирования Kotlin, являющийся основным и официально поддерживаемым языком для разработки под операционную систему Android. Его использование обеспечивает высокую совместимость с современными инструментами Android-платформы, а также позволяет создавать стабильные, безопасные и эффективные приложения [4].

Одним из ключевых преимуществ Kotlin является его тесная интеграция с Jetpack Compose – современной декларативной технологией для построения пользовательских интерфейсов. Compose предоставляет высокий уровень абстракции, снижает количество шаблонного кода и позволяет быстрее создавать гибкие и адаптивные интерфейсы. Поскольку приложение включает несколько динамически изменяемых экранов (поиск, избранное, рекомендации, профиль, детали материала), возможность реактивного обновления интерфейса является критически важной [5].

Кроме того, Kotlin обладает рядом встроенных возможностей, упрощающих разработку сложных приложений:

- поддержка корутин, обеспечивающая удобную и безопасную работу с асинхронными операциями, такими как сетевые запросы к внешнему API [6];
- высокая безопасность типов, уменьшающая вероятность ошибок, связанных с обработкой null-значений [7];

- совместимость с Java, что позволяет при необходимости интегрировать существующие библиотеки и решения [4];
- выраженный лаконичный синтаксис, который ускоряет разработку и повышает читаемость кода [7].

Язык также хорошо подходит для реализации архитектурных подходов MVVM и работы с библиотеками Android Jetpack, которые используются в проекте – ViewModel, Room, Navigation Compose и др. Такой выбор обеспечивает структурированность разработки и повышает масштабируемость и поддерживаемость приложения.

Jetpack Compose – современный UI-фреймворк для Android, который предоставляет декларативный подход к описанию интерфейса: разработчик описывает что должно отображаться, а фреймворк сам определяет как это отрисовать. Такой подход снижает вероятность ошибок, уменьшает объём шаблонного кода и делает интерфейс проще в сопровождении. Compose нативно интегрирован с Kotlin, поддерживает корутины и State-модели, что особенно важно для динамичных экранов с сетевыми запросами и локальными базами данных.

MVVM и ViewModel обеспечивают чёткое разделение логики и интерфейса:

- model отвечает за данные и бизнес-логику;
- viewModel управляет состоянием экрана и взаимодействием с моделью;
- view (Compose-UI) занимается только отображением данных.

ViewModel сохраняет состояние при изменении конфигурации устройства (например, при повороте экрана) и позволяет тестировать бизнес-логику независимо от интерфейса. Это делает приложение модульным, устойчивым и легко масштабируемым.

Room используется для безопасного и стабильного локального хранения данных. Эта библиотека предоставляет удобный API для работы с SQLite,

поддерживает сущности, DAO, миграции и автоматические проверки запросов. Room интегрируется с корутинами, ViewModel и Flow, что позволяет выполнять операции в фоне без блокировки интерфейса. Она обеспечивает сохранение избранных материалов пользователя и корректную работу приложения даже при отсутствии интернет-соединения.

Navigation Compose управляет маршрутизацией между экранами, полностью интегрирован с Jetpack Compose и MVVM. Он позволяет декларативно описывать переходы, хранить состояние экранов, передавать параметры и управлять стеком экранов. Такой подход упрощает добавление новых экранов и поддерживает чистую архитектуру.

Таким образом, использование Kotlin в сочетании с архитектурой MVVM и библиотеками Android Jetpack (ViewModel, Room, Navigation Compose, Jetpack Compose) обосновано современностью языка, его удобством для разработки, надёжностью и поддержкой передовых инструментов Android. Такой подход позволяет создавать масштабируемое, поддерживаемое и безопасное приложение с динамическими экранами и взаимодействиями с удалёнными API, обеспечивая при этом стабильное хранение данных и обновление интерфейса.

3.2 Обоснование выбора среды разработки

Для разработки мобильного приложения под Android в рамках проекта в качестве среды разработки была выбрана Android Studio. Это решение оправдано рядом существенных преимуществ, которые делают Android Studio оптимальным инструментом для задач, поставленных в проекте:

- официальная среда для Android. Android Studio — официальная интегрированная среда разработки (IDE) для платформы Android; она полностью поддерживает работу с Kotlin (и Java) и соответствует стандартам разработки Android-приложений [8];

- мощный и полный набор инструментов. Среда предоставляет

					KP.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		23

редактор кода, средства отладки, профилирования, систему сборки Gradle, инструменты управления ресурсами, макетами и пользовательским интерфейсом. Всё это даёт возможность полноценно разрабатывать, тестировать и оптимизировать приложение без необходимости подключать множество сторонних утилит;

- поддержка современных инструментов и библиотек Android. Android Studio нативно поддерживает Kotlin и все актуальные Android-библиотеки, что особенно важно при использовании подходов вроде MVVM, Jetpack Compose, Room и др. [9];

- удобство работы с разными устройствами и конфигурациями. В составе Android Studio есть встроенный эмулятор, что облегчает тестирование приложения на разных устройствах, экранах и версиях Android без необходимости наличия физического устройства;

- широкое сообщество, стабильность и регулярные обновления. Поскольку Android Studio – стандартный инструмент для Android-разработки, существует большое сообщество, большое количество документации, примеров, устойчивый цикл обновлений, что снижает риск проблем совместимости и повышает надёжность разработки.

Таким образом, Android Studio – это полноценная, мощная и официально рекомендованная среда для разработки Android-приложений. Благодаря её функционалу, стабильности и совместимости с Kotlin и современными Android-библиотеками, она является логичным и обоснованным выбором для разработки данного мобильного приложения, обеспечивая удобство, эффективность и поддержку при реализации всей необходимой функциональности.

3.3 Описание структуры программного средства

Проект разделён на несколько ключевых директорий и компонентов, что обеспечивает удобство разработки и сопровождения кода.

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		24

Рассмотрим основные директории:

- data/ – содержит модели данных, DAO для работы с локальной базой Room и репозитории для взаимодействия с API и базой данных;
- network/ – реализует работу с внешним API, включая сетевые запросы и обработку ответов;
- screens/ – экранные компоненты приложения на Jetpack Compose, отвечающие за отображение интерфейса пользователю;
- navigation/ – релизует навигацию приложения.

Также основными компонентами приложения являются:

- app.kt – управляет жизненным циклом приложения и настройкой локальной базы с безопасными миграциями;
- authState.kt – управляет логикой входа/выхода и уведомляет UI о состоянии авторизации;
- mainActivity.kt – управляет UI, темой приложения и навигацией, связывая всё с состоянием пользователя (AuthState) и базой данных (App.kt).

Такая структура проекта способствует эффективной разработке, обеспечивает высокое качество кода, что особенно важно для долгосрочных и масштабируемых проектов.

3.3.1 Класс «MainActivity»

Класс MainActivity отвечает за запуск пользовательского интерфейса разрабатываемого приложения, инициализацию темы темной или светлой через системные настройки устройства или специальный параметр ThemePreferences, связывание состояния аутентификации и настройку навигации между экранами. Он интегрируется с классом AuthState для управления состоянием пользователя и использует Jetpack Compose для построения UI.

В таблице 3.1 представлены основные методы и функционал класса.

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		25

Таблица 3.1 – Методы класса «MainActivity»

Метод	Входные параметры	Описание	Возвращаемое значение
1	2	3	4
onCreate	savedInstanceState: Bundle? – это параметр метода onCreate(), который позволяет восстановить состояние Activity после пересоздания, например, при повороте экрана или возвращении из фона. Если данных для восстановления нет, параметр равен null	Основной метод жизненного цикла активности. Инициализирует AuthState, устанавливает тему приложения (тёмную или светлую), обеспечивает обработку системной темы и прокидывает навигацию через SetupNavController	-

Описание ключевых функций и действий внутри onCreate:

- получение экземпляра AuthState через ViewModelProvider для контроля состояния пользователя (вход/выход);
- определение текущей темы через ThemePreferences или системную тему (isSystemInDarkTheme);
- реализация механизма обновления темы через CompositionLocalProvider и LocalThemeUpdater;
- оборачивание UI в StudentMaterialsSearchTheme и Surface для корректного отображения цвета фона и элементов интерфейса;
- вызов SetupNavController(authState) для запуска навигации между экранами, включая передачу состояния аутентификации для контроля доступа.

Класс MainActivity обеспечивает запуск приложения, настройку интерфейса, привязку состояния пользователя и управление навигацией,

связывая базовую логику приложения с UI.

3.3.2 Класс «App»

Класс App представляет собой класс приложения, отвечающий за инициализацию базы данных Room, настройку миграций и создание глобального экземпляра приложения. Он обеспечивает безопасное обновление структуры базы данных и доступ к ней из любой части приложения.

В таблице 3.2 представлены основные методы и свойства класса.

Таблица 3.2 – Методы класса «App»

Метод	Входные параметры	Описание	Возвращаемое значение
1	2	3	4
database	-	Ленивая инициализация базы данных Room с именем student_app.db. Подключает миграции и временно разрешает выполнение запросов в основном потоке для упрощения разработки	AppDatabase – база данных
instance	-	Статический экземпляр приложения для глобального доступа	App-экземпляр приложения
onCreate()	-	Жизненный цикл приложения. Инициализирует глобальный экземпляр и запускает инициализацию базы данных в фоне через GlobalScope.launch(Dispatchers.IO)	-

3.3.3 Класс «AuthState»

Класс AuthState отвечает за хранение и управление текущим состоянием аутентификации пользователя. Он реализован как ViewModel и предоставляет реактивное состояние (StateFlow) для компонентов UI, что позволяет автоматически обновлять интерфейс при входе или выходе пользователя.

В таблице 3.3 представлены основные методы и свойства класса.

Таблица 3.3 – Методы класса «AuthState»

Метод	Входные параметры	Описание	Возвращаемое значение
1	2	3	4
_currentUser	-	Приватное MutableStateFlow, хранит текущего пользователя (или null, если пользователь не авторизован)	MutableStateFlow<User?>
currentUser	-	Публичное StateFlow для доступа к текущему пользователю.	StateFlow<User?>
isLoggedIn	-	Показывает, авторизован ли пользователь.	StateFlow<Boolean>
login(user: User)	user: User – пользователь	Выполняет вход пользователя, обновляя _currentUser	-
logout()	-	Выполняет выход пользователя, обнуляя _currentUser	-

3.3.4 Класс «SetupNavGraph»

Класс SetupNavGraph отвечает за конфигурацию навигации внутри мобильного приложения и обеспечивает маршрутизацию между экранами на основе текущего состояния авторизации пользователя. Он определяет доступные маршруты, связывает их с соответствующими интерфейсами и управляет поведением нижней навигационной панели. Навигация реализована с использованием библиотеки Jetpack Navigation Compose, что позволяет описывать структуру экранов декларативно и гибко.

Класс также отслеживает поток состояния авторизации и автоматически переключает стартовый экран между Login и Search. При успешной авторизации выполняется очистка стека возврата, чтобы исключить возврат на экран входа. Для передачи данных между экранами, например URL-адреса в детализации или WebView, используется механизм безопасной навигации с параметрами маршрутов.

В таблице 3.4 представлены основные методы и элементы класса SetupNavGraph.

Таблица 3.4 – Методы класса «SetupNavGraph»

Метод	Входные параметры	Описание	Возвращаемое значение
1	2	3	4
SetupNavGraph	authState: AuthState, navController: NavController	Основной компонент, создающий граф навигации, определяющий стартовый экран, связи между маршрутами и логику переходов	-
Screen.Details. createRoute(url)	url: String	Кодирует URL для передачи через параметры навигации и формирует корректный маршрут	String – готовый маршрут

Окончание таблицы 3.4

1	2	3	4
Screen	-	Описывает все доступные экраны приложения, включая их маршруты, названия и иконки. Содержит вложенные объекты: Login, Search, Favorites, Recommendations, Account, Details, WebView	Набор контролируемых маршрутов
Screen.WebView.createRoute(url)	url: String	Кодирование и безопасная передача URL для просмотра страницы внутри приложения	String – готовый маршрут
BottomNavBar	navController: NavHostController, isLoggedIn: Boolean, currentRoute: String?	Генерирует нижнюю панель навигации, определяет доступные вкладки в зависимости от авторизации, выделяет текущую страницу	-
NavHost	Параметры навигации + список composable-функций	Формирует дерево доступных маршрутов и связывает каждый маршрут с соответствующим экраном	-

3.4 Выводы по главе

В данной главе были рассмотрены ключевые решения, определяющие архитектуру и технологическую основу разрабатываемого мобильного приложения. На основе анализа современных инструментов Android-разработки обоснован выбор языка Kotlin, архитектурного подхода MVVM и набора библиотек Android Jetpack, таких как ViewModel, Room, Navigation

Compose и Jetpack Compose. Эти технологии обеспечивают высокую эффективность разработки, надёжность работы и удобство сопровождения приложения.

Было также обосновано использование Android Studio как основной среды разработки – благодаря её интеграции с Kotlin, встроенными инструментами отладки, профилирования и тестирования, а также полноценной поддержкой всех применяемых фреймворков и библиотек.

Кроме того, в главе подробно описана структура проекта и назначение основных программных компонентов. Рассмотрены внутреннее устройство приложения, взаимосвязи между слоями данных, навигацией и пользовательским интерфейсом. Особое внимание уделено ключевым классам – MainActivity, App, AuthState и SetupNavGraph, которые обеспечивают запуск, хранение состояния, авторизацию пользователя и маршрутизацию между экранами.

Таким образом, проведённое программное конструирование подтверждает, что выбранная архитектура является устойчивой, масштабируемой и полностью соответствует требованиям проекта. Она позволяет реализовать все необходимые функции приложения, обеспечивает высокое качество кода и создаёт основу для дальнейшего развития системы.

4 Проверка работоспособности программного средства

В данном разделе описаны основные сценарии использования приложения поиска и персонализации образовательных материалов, описан процесс их проверки и представлен результат выполнения проверок на различных входных данных.

4.1 Описание сценариев использования

Проверка работоспособности разработанного мобильного приложения проводится с целью убедиться в корректности реализации основных функций, устойчивости работы интерфейса и соответствия поведения системы заявленным требованиям. Проверка включает выявление ошибок, оценку удобства использования, корректности отображения данных и стабильности выполнения пользовательских сценариев.

Тестирование будет осуществляться по стратегии «черного ящика», что позволит оценивать работу системы исключительно с позиции конечного пользователя, без анализа внутренней структуры кода. Такой подход обеспечивает объективную проверку функционала приложения и выявление несоответствий между ожидаемым и реальным поведением интерфейса.

Для подтверждения корректности работы будут рассмотрены ключевые сценарии использования:

- поиск материалов по ключевым словам. Проверяется корректность обработки поискового запроса, отображение найденных материалов, реакция на пустой запрос и отсутствие результатов;
- просмотр подробной информации о материале. Оценивается корректность загрузки карточки результата, отображение заголовка, сниппета, URL-адреса и возможность открытия материала в браузерном окне;
- добавление и удаление материалов из избранного. Проверяются корректность сохранения элемента, его отображение в разделе избранного и

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		32

дальнейшее удаление;

- формирование персональных рекомендаций. Проверяется корректность отображения предложенных запросов, сформированных на основе интересов пользователя и анализа его избранного;

- работа с профилем пользователя. Проверяется корректность загрузки данных профиля, изменение имени, пароля, списка интересов, а также работа переключения светлой и темной темы.

Каждый сценарий включает последовательность шагов, ожидаемый результат и фактическое поведение системы. На основании анализа сценариев будет подтверждена корректность реализации ключевых функций приложения и его готовность к использованию конечными пользователями.

4.2 Сценарий «Поиск материалов по ключевым словам»

Цель данного сценария – убедиться в корректной работе основного функционала приложения – поиска образовательных материалов с использованием API SerpStack, отображения найденных результатов, обработки ошибок.

После запуска приложения пользователь может перейти на страницу «Поиск», а после успешной авторизации пользователь автоматически попадает на страницу «Поиск», представленную на рисунке 4.1. Интерфейс экрана включает:

- поле ввода поискового запроса;
- кнопку запуска поиска;
- область отображения найденных материалов;
- обработку состояния загрузки и ошибок.

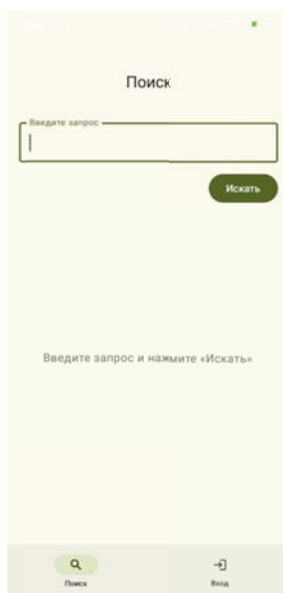


Рисунок 4.1 – Страница поиска материалов

Чтобы выполнить поиск, пользователь вводит ключевое слово или фразу (например: «kotlin coroutines tutorial») в текстовое поле и нажимает кнопку «Искать» либо клавишу Search на клавиатуре.

Если материалы найдены, пользователь видит список карточек, это можно увидеть на рисунке 4.2. Каждая карточка содержит:

- заголовок найденной страницы;
- краткое описание (сниппет);
- URL-адрес источника.

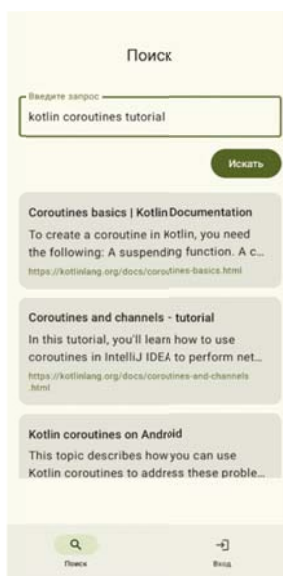


Рисунок 4.2 – Отображение результатов поиска

При нажатии на карточку результата выполняются два действия:

- материал автоматически добавляется в историю текущего пользователя, если он авторизован;
- открывается экран деталей, где отображается подробная информация о выбранном материале.

Если пользователь выполняет пустой запрос, кнопка поиска становится недоступной, что предотвращает отправку некорректного запроса.

Если API возвращает ошибку или отсутствуют результаты, приложение отображает сообщение об ошибке либо уведомление «Ничего не найдено», позволяя пользователю изменить запрос.

Таким образом, проведённая проверка подтвердила корректность выполнения основного сценария поиска:

- запрос правильно обрабатывается;
- результаты отображаются корректно;
- интерфейс правильно реагирует на ошибки, пустые запросы и отсутствие данных.

4.3 Сценарий «Просмотр подробной информации о материале»

Данный сценарий позволяет убедиться в корректности отображения полной информации о выбранном пользователем материале, а также проверяет работу кнопок взаимодействия:

- добавление в избранное;
- открытие страницы в приложении;
- открытие в стороннем браузере.

После выбора карточки из результатов поиска пользователь переходит на экран подробной информации, представленной на рисунке 4.3 для авторизованного пользователя и на рисунке 4.4 – для не авторизованного.

Экран содержит:

- заголовок материала;

					KP.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		35

- краткое описание (сниппет);
- кнопку «Добавить в избранное» / «Удалить из избранного»;
- кнопку «Открыть в приложении» для перехода к WebView;
- кнопку «Открыть в браузере» для открытия материала в стороннем браузере.

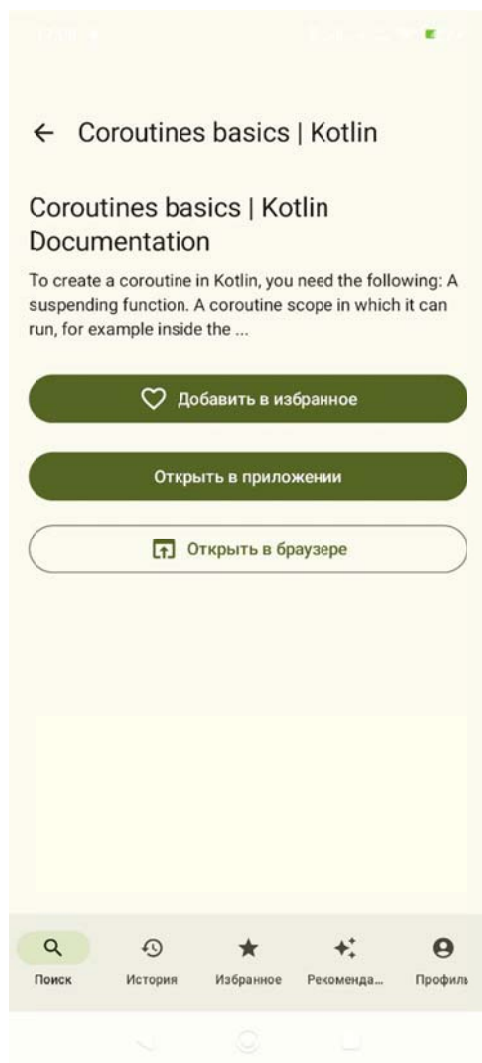


Рисунок 4.3 – Экран подробной информации после авторизации

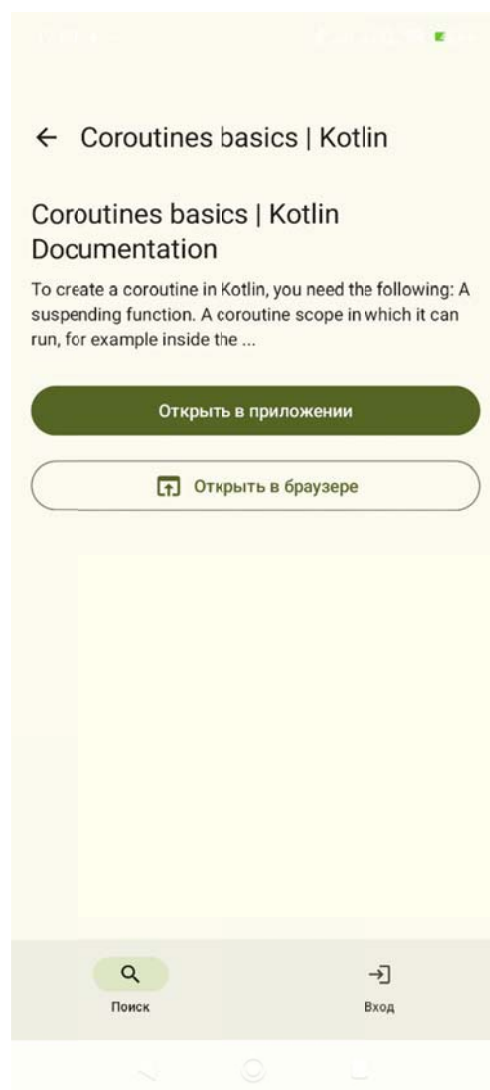


Рисунок 4.4 – Экран подробной информации без авторизации

При загрузке экрана приложение:

- запрашивает в базе данных информацию, был ли материал ранее добавлен в избранное;
- автоматически сохраняет материал в раздел История, если пользователь авторизован;

- отображает индикатор загрузки до появления данных.

Пользователь может просмотреть контент, вернуться назад или перейти на страницу источника. Все функции срабатывают корректно:

- кнопки активны;
- навигация работает;
- визуальные элементы отображаются в соответствии с выбранной темой (светлой или тёмной).

Таким образом, проверка подтвердила, что функционал подробного просмотра материалов работает стабильно и корректно.

4.4 Сценарий «Добавление материала в избранное»

Рассмотрим процесс добавления товара в корзину.

Цель данного сценария – убедиться, что пользователь может вручную сохранять материалы в избранное и удалять их из него.

Добавление в избранное выполняется только по нажатию кнопки на экране подробной информации. Функция доступна только после авторизации. Автоматическое сохранение отсутствует.

Чтобы проверить корректность работы, пользователь переходит в раздел «Избранное», где отображается список сохранённых ранее материалов, который отображается на рисунке 4.5. Каждый элемент содержит:

- заголовок;
- краткое описание;
- ссылку на источник.

При нажатии на карточку открывается экран подробностей выбранного материала.



Рисунок 4.5 – Избранное до добавления нового материала

Для добавления найденного на предыдущем шаге материала пользователь выполняет следующие действия:

- переходит к подробному просмотру материала, который изображен на рисунке 4.3;
- нажимает кнопку «Добавить в избранное».

Кнопка меняет свое состояние на «Удалить из избранного», отображая статус, это показано на рисунке 4.6.

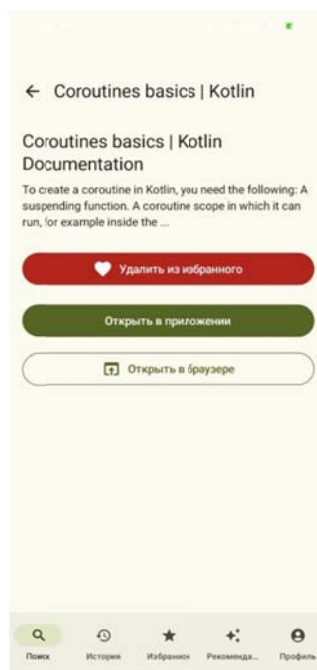


Рисунок 4.6 – Добавление ранее найденного материала

Пользователь переходит в раздел «Избранное» для проверки, что новый материал отображается. Результат представлен на рисунке 4.7.



Рисунок 4.7 – Раздел «Избранное» после добавления нового материала

Удаление выполняется повторным нажатием кнопки на странице материала. После удаления материал исчезает из списка без перезагрузки страницы.

Проверка подтвердила корректность логики:

– избранное работает стабильно;

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		39

- данные привязаны к конкретному пользователю;
- отображение обновляется мгновенно.

4.5 Сценарий «Формирование персональных рекомендаций»

Рассмотрим процесс оформления заказа.

Цель сценария — убедиться, что приложение корректно формирует и отображает персональные запросы, предлагаемые пользователю на основе его интересов и материалов, сохранённых в избранное.

Раздел «Рекомендации» представлен на рисунке 4.7 и содержит список карточек с предложенными поисковыми запросами. Система формирует рекомендации на основе:

- интересов пользователя, указанных в профиле;
- ключевых слов, встречающихся в избранных материалах.

Каждый рекомендованный запрос формируется, фильтруется от лишних символов, очищается от чисел и коротких слов, затем перемешивается и отображается пользователю.

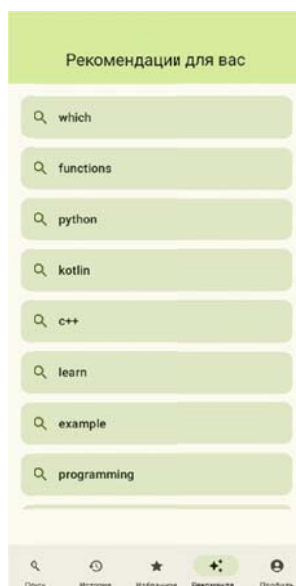


Рисунок 4.7 – Раздел «Рекомендации»

При нажатии на карточку:

- пользователь автоматически переносится на страницу «Поиск»;

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		40

- запрос подставляется в строку поиска и запускается автоматически.

Проверка показала:

- рекомендации формируются корректно;
- поиск по рекомендованному запросу запускается без ручного ввода;
- пустые рекомендации корректно обрабатываются (приложение сообщает, что необходимо добавить интересы или избранное).

Таким образом, механизм рекомендаций работает корректно и повышает удобство использования приложения.

4.6 Сценарий «Работа с профилем пользователя»

Раздел «Профиль» позволяет пользователю просматривать и изменять свои персональные данные, управлять интересами, переключать тему приложения и выполнять выход из аккаунта. Данный сценарий проверяет корректность работы всех функций экрана, включая загрузку данных, редактирование, сохранение и обработку изменения состояния авторизации.

После запуска приложения пользователь проходит процедуру авторизации на экране входа, который показан на рисунке 4.8. Для этого он вводит email и пароль и нажимает кнопку «Войти/Зарегистрироваться». В случае корректного заполнения формы:

- пользователь видит сообщение «Добро пожаловать!» – если аккаунт существует, и сообщение «Регистрация успешна! Добро пожаловать!» - если создан новый аккаунт;
- пользователь перенаправляется на главный экран поиска.

После авторизации становится доступна навигация к разделу «Профиль» через нижнее меню приложения.

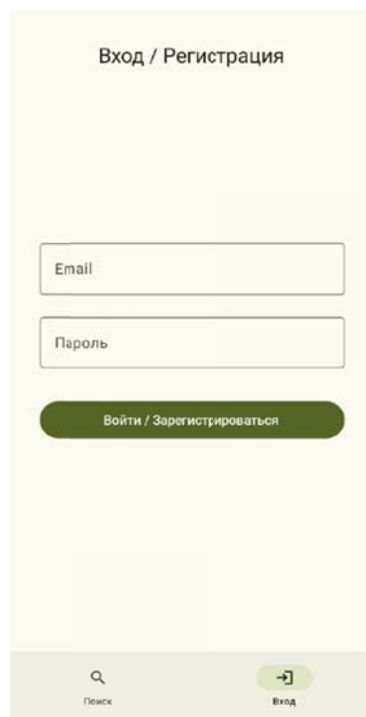


Рисунок 4.8 – Экран авторизации

При переходе к разделу «Профиль» пользователь видит:

- аватар;
- приветствие;
- блок с email;
- блок с интересами;
- кнопка «Выйти из аккаунта»;
- переключатель темы.

Интерфейс обновляется мгновенно — данные появляются без необходимости перезагружать экран. Раздел профиля пользователя представлен на рисунке 4.9.

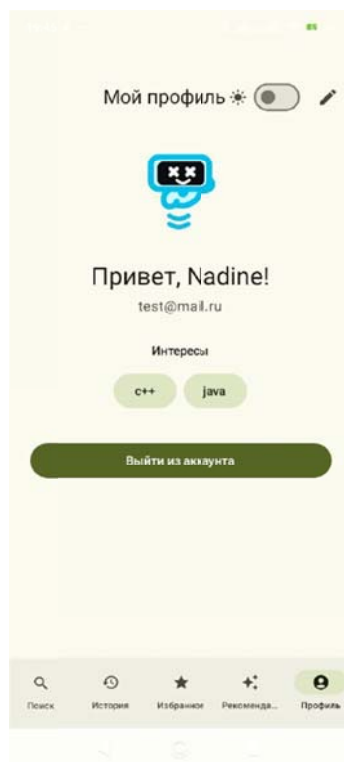


Рисунок 4.9 – Профиль пользователя

Для редактирования профиля пользователь нажимает иконку «Редактировать» в правом верхнем углу.

После нажатия пользователь переходит в режим редактирования, который изображен на рисунке 4.10:

- отображение имени и email заменяется текстовыми полями;
- становится доступным ввод имени;
- поле email остаётся неактивным, чтобы предотвратить нарушение связи с данными;
- появляется поле для ввода нового пароля;
- отображается форма добавления интересов.

Пользователь изменяет имя, нажимает кнопку «Сохранить», после чего имя в профиле изменяется. Это видно на рисунке 4.11, имя пользователя «Nadine» изменилось на «Надежда».

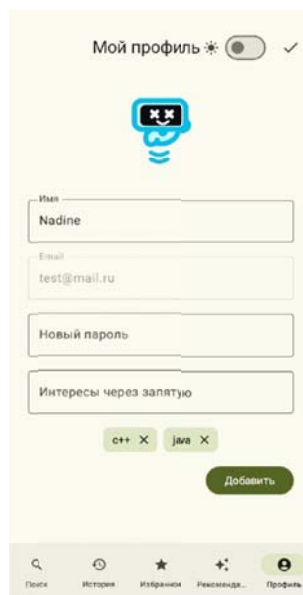


Рисунок 4.10 – Редактирование профиля

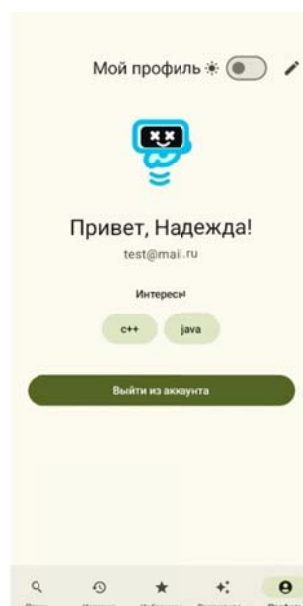


Рисунок 4.11 – Профиль пользователя с новым именем

Пользователь может ввести новое значение в поле «Новый пароль» и сохраняет изменения. После сохранения новый пароль начинает использоваться при следующей авторизации.

Добавление интереса:

- пользователь вводит интерес, например kotlin;
- нажимает кнопку «Добавить».

После сохранения интересы отображаются в профиле.

Чтобы удалить интерес пользователю нужно нажать на крестик у

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		44

соответствующего интереса в режиме редактирования.

На экране профиля находится переключатель для смены темы. При его нажатии тема приложения меняется с светлой на темную и наоборот. Темная тема представлена на рисунке 4.12.

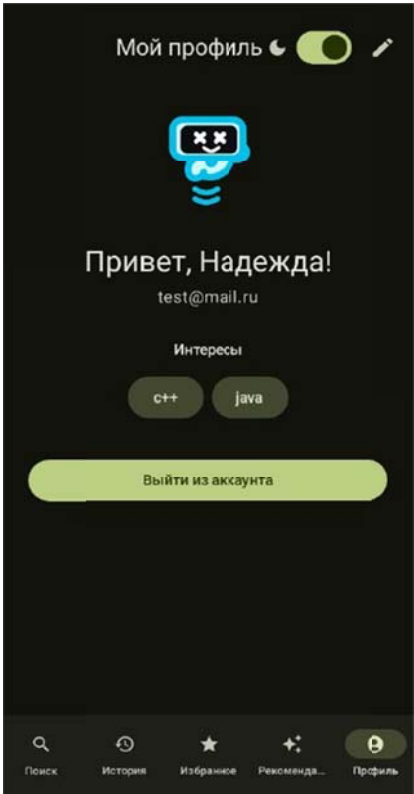


Рисунок 4.12 – Темная тема приложения

При нажатии кнопки «Выйти из аккаунта» пользователь перенаправляется на страницу авторизации.

Все функции раздела «Профиль» работают корректно:

- загрузка данных – стабильная;
- изменения имени и пароля – сохраняются без перезагрузки;
- интересы – корректно добавляются, удаляются и влияют на рекомендации;
- переключатель темы – мгновенно обновляет интерфейс и запоминает состояние;
- выход из аккаунта – корректно сбрасывает сессию и возвращает пользователя к экрану входа.

Раздел полностью готов к использованию пользователями и отвечает функциональным требованиям приложения.

4.7 Выводы по главе

В результате проведённой проверки работоспособности мобильного приложения были рассмотрены ключевые пользовательские сценарии, обеспечивающие основной функционал системы, такие как поиск материалов, просмотр подробной информации, работа с избранным, формирование персональных рекомендаций и управление профилем пользователя. Все тесты выполнялись по стратегии «чёрного ящика», что позволило оценить поведение интерфейса исключительно с позиции конечного пользователя.

Проверка показала, что приложение корректно обрабатывает поисковые запросы, отображает результаты и реагирует на различные состояния, включая отсутствие данных и ошибки API. Экран подробной информации надёжно загружает данные о материале, обеспечивает навигацию и предоставляет доступ к дополнительным функциям, таким как открытие в браузере и добавление в избранное.

Функционал избранного работает стабильн – материалы сохраняются только вручную, корректно привязываются к конкретному пользователю и отображаются без задержек.

Механизм формирования рекомендаций показал корректность анализа интересов и избранных материалов, а также удобство автоматического подстановки запросов.

Раздел «Профиль» подтвердил устойчивую работу всех элементов – данные загружаются корректно, изменения имени, пароля и интересов сохраняются в базе данных, переключение темы происходит мгновенно, а выход из аккаунта надёжно сбрасывает состояние пользователя. Интерфейс реагирует быстро, без необходимости перезагрузки экранов.

По итогам тестирования можно сделать вывод, что разработанное

					<i>KP.820000.000</i>	Лист
Изм.	Лист	№ докум.	Подпись	Дата		46

приложение демонстрирует стабильную работу основных функций, соответствует требованиям, предъявляемым к системе, и готово к использованию конечными пользователями. Проведённая проверка не выявила критических ошибок.

Заключение

В ходе выполнения данной работы было разработано мобильное приложение для поиска и персонализации образовательных материалов, обеспечивающее пользователю удобный доступ к учебным ресурсам по ключевым словам. Приложение предоставляет возможность просматривать найденные материалы, открывать подробную информацию, сохранять понравившиеся элементы в избранное, формировать персональные рекомендации на основе интересов, а также управлять данными профиля. Все основные функции реализованы в соответствии с установленными требованиями и обеспечивают стабильную, корректную и удобную работу приложения.

Данное программное средство полностью реализует задачи, поставленные в пункте 1.2, а именно:

- разработать алгоритмы, обеспечивающие обработку данных о товарах, включая сортировку и фильтрацию по категориям и видам животных;
- выбрать подходящий язык программирования и платформу для реализации;
- сформировать структуру хранения данных с использованием локального хранилища;
- реализовать интерфейс для каталога товаров с возможностью перехода на страницы подробного описания продуктов;
- создать модуль корзины покупок, позволяющий пользователю редактировать количество выбранных товаров, просматривать итоговую стоимость, выбирать товары для оформления заказа и проводить безопасный процесс оплаты;
- провести тестирование приложения, выявить и устранить возможные уязвимости.
- спроектировать архитектуру приложения, определив основные модули, такие как поиск + API-запросы, база данных, интерфейс, авторизация,

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		48

хранение пользовательских данных, навигация;

- реализовать функционал регистрации и авторизации пользователей;
- интегрировать внешний API для поиска контента, настроить механизм запроса, получения и отображения результатов;
- реализовать интерфейс для поиска и отображения результатов, включая поиск по ключевым словам, отображение списка, переход к подробностям, чтобы пользователь мог удобно просматривать найденные материалы;
- организовать хранение данных локально, чтобы пользователь мог сохранять материалы «в избранное» и обращаться к ним офлайн;
- добавить систему рекомендаций, базирующуюся на истории поиска или сохранённых материалах пользователя — чтобы приложение могло предлагать релевантные ресурсы;
- реализовать навигацию и удобный UI/UX, обеспечивающий комфортную работу;
- провести тестирование приложения, проверить корректность работы всех функций.

Получены и закреплены навыки разработки мобильного приложения на языке программирования Kotlin с использованием фреймворка Jetpack Compose, включая создание экранов и UI-компонентов, работу с состояниями и ViewModel, интеграцию с внешним API, взаимодействие с локальной базой данных, реализацию навигации и персонализации пользовательского опыта.

Перечень использованных информационных ресурсов

1. What Are The Advantages of Smartphone in Education? An In-depth Look – HONOR ZA [Электронный ресурс], URL: <https://www.honor.com/za/blog/advantages-of-smartphone-in-education/> (дата обращения: 06.12.2025);
2. Почему мобильное обучение онлайн-курсам важнее десктопа в 2025 году — Okumy [Электронный ресурс], URL: https://okumy.com/blog/akv_pochemu-mobilnoe-obuchenie-onlajn-kursam-vazhnee-desktopa-v-2025-godu/ (дата обращения: 06.12.2025);
3. Importance of Mobile Apps for Educational Institutions | MoldStud [Электронный ресурс], URL: <https://moldstud.com/articles/p-why-mobile-app-development-is-essential-for-educational-institutions-an-in-depth-overview> (дата обращения: 06.12.2025);
4. Kotlin for Android | Kotlin Documentation [Электронный ресурс], URL: <https://kotlinlang.org/docs/android-overview.html> (дата обращения: 09.12.2025);
5. Kotlin for Jetpack Compose | Android Developers [Электронный ресурс], URL: <https://developer.android.com/develop/ui/compose/kotlin> (дата обращения: 09.12.2025);
6. Kotlin and Android | Android Developers [Электронный ресурс], URL: <https://developer.android.com/kotlin> (дата обращения: 09.12.2025);
7. Build Better Apps with Kotlin | Android Developers [Электронный ресурс], URL: <https://developer.android.com/kotlin/build-better-apps> (дата обращения: 09.12.2025);
8. Как писать приложения под Android / Хабр [Электронный ресурс], URL: <https://habr.com/ru/articles/797609/> (дата обращения: 09.12.2025);
9. Как выбрать язык программирования для создания Андроид — приложения / Хабр [Электронный ресурс], URL: <https://habr.com/ru/articles/477578> (дата обращения: 09.12.2025).

Приложение А Листинг программы

Листинг А.1 – Класс «App.kt»

```
package nadinee.studentmaterialssearch

import android.app.Application
import androidx.room.Room
import androidx.room.migration.Migration
import androidx.sqlite.db.SupportSQLiteDatabase
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch
import nadinee.studentmaterialssearch.data.AppDatabase

class App : Application() {
    companion object {
        val database: AppDatabase by lazy {
            Room.databaseBuilder(
                instance.applicationContext,
                AppDatabase::class.java,
                "student_app.db"
            )
                .addMigrations(MIGRATION_1_2, MIGRATION_2_3,
MIGRATION_3_4, MIGRATION_4_5)
                .fallbackToDestructiveMigration()
                .allowMainThreadQueries()
                .build()
        }
        val MIGRATION_1_2 = object : Migration(1, 2) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("ALTER TABLE users ADD COLUMN name
TEXT NOT NULL DEFAULT 'Без имени'")
                db.execSQL("ALTER TABLE users ADD COLUMN
interests TEXT NOT NULL DEFAULT ''")
            }
        }
        val MIGRATION_2_3 = object : Migration(2, 3) {
```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		51

```

        override fun migrate(db: SupportSQLiteDatabase) {
            db.execSQL("""
                CREATE TABLE IF NOT EXISTS favorites (
                    url TEXT PRIMARY KEY NOT NULL,
                    title TEXT NOT NULL,
                    content TEXT NOT NULL,
                    addedAt INTEGER NOT NULL DEFAULT 0
                )
            """).trimIndent())
        }
    }

    val MIGRATION_3_4 = object : Migration(4, 5) {
        override fun migrate(db: SupportSQLiteDatabase) {
            // Создаём новую таблицу
            db.execSQL("""
                CREATE TABLE favorites_new (
                    url TEXT PRIMARY KEY NOT NULL,
                    userEmail TEXT NOT NULL DEFAULT
'unknown@user.ru',
                    title TEXT NOT NULL,
                    content TEXT NOT NULL,
                    addedAt INTEGER NOT NULL DEFAULT 0
                )
            """).trimIndent()

            db.execSQL("""
                INSERT INTO favorites_new (url,
userEmail, title, content, addedAt)
                SELECT
                    f.url,
                    COALESCE(u.email, 'legacy@user.ru'),
                    f.title,
                    f.content,
                    COALESCE(f.addedAt, 0)
                FROM favorites f
            """)
        }
    }

```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		52

```

        LEFT JOIN users u ON 1=1
        LIMIT 1000
        """.trimIndent())

    // Удаляем старую
    db.execSQL("DROP TABLE IF EXISTS favorites")
    db.execSQL("ALTER TABLE favorites_new RENAME
TO favorites")

    db.execSQL("CREATE INDEX IF NOT EXISTS
index_favorites_userEmail ON favorites(userEmail)")
    }
}

val MIGRATION_4_5 = object : Migration(4, 5) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("""
CREATE TABLE history (
    id INTEGER PRIMARY KEY AUTOINCREMENT NOT
NULL,

    userEmail TEXT,
    url TEXT NOT NULL,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    viewedAt INTEGER NOT NULL

)
        """.trimIndent())
        db.execSQL("CREATE INDEX
index_history_userEmail ON history(userEmail)")
    }
}

lateinit var instance: App
private set

}

override fun onCreate() {
    super.onCreate()
    instance = this
}

```

```

        GlobalScope.launch(Dispatchers.IO) {
            database
        }
    }
}

```

Листинг А.2 – Класс « AuthState.kt»

```

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch
import nadinee.studentmaterialssearch.data.User

class AuthState : ViewModel() {
    private val _currentUser = MutableStateFlow<User?>(null)
    val currentUser: StateFlow<User?> =
        _currentUser.asStateFlow()
    val isLoggedIn: StateFlow<Boolean> = _currentUser
        .asStateFlow()
        .map { it != null }
        .stateIn(viewModelScope, SharingStarted.Eagerly,
false)

    fun login(user: User) {
        viewModelScope.launch {
            _currentUser.emit(user)
        }
    }

    fun logout() {
        viewModelScope.launch {
            _currentUser.emit(null)
        }
    }
}

```

```
}
```

Листинг А.3 – Класс «MainActivity.kt»

```
package nadinee.studentmaterialssearch

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.lifecycle.ViewModelProvider
import nadinee.studentmaterialssearch.data.ThemePreferences
import

nadinee.studentmaterialssearch.navigation.SetupNavGraph

import nadinee.studentmaterialssearch.ui.theme.LocalAppTheme
import

nadinee.studentmaterialssearch.ui.theme.LocalThemeUpdater

import

nadinee.studentmaterialssearch.ui.theme.StudentMaterialsSearchThem
e

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val authState =
ViewModelProvider(this) [AuthState::class.java]

        setContent {
            val context = LocalContext.current
            val systemIsDark = isSystemInDarkTheme()
```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		55

```

        var isDark by remember {
mutableStateOf (ThemePreferences.getTheme(context) ?: systemIsDark)
    }

    LaunchedEffect(systemIsDark) {
        if (ThemePreferences.getTheme(context) ==
null) {

            isDark = systemIsDark

        }

    }

    CompositionLocalProvider(
        LocalThemeUpdater provides { newValue ->
            isDark = newValue
            ThemePreferences.setDarkTheme(context,
newValue)

        }
    ) {

        StudentMaterialsSearchTheme(isDark = isDark)
    {

        Surface(
            modifier = Modifier.fillMaxSize(),
            color =
MaterialTheme.colorScheme.background
        ) {

            SetupNavGraph(authState = authState)

        }

    }

    }

}

```

Листинг А.4 – Класс « SetupNavGraph.kt»

```

package nadinee.studentmaterialssearch.navigation

import AuthState

import androidx.compose.foundation.layout.padding

```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		56


```

import androidx.compose.foundation.layout.size
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.*
import androidx.navigation.compose.*
import kotlinx.coroutines.flow.collectLatest
import nadinee.studentmaterialssearch.screens.*
import java.net.URLDecoder
import java.net.URLEncoder

sealed class Screen(val route: String, val title: String, val
icon: ImageVector? = null) {
    object Login : Screen("login", "Вход",
Icons.Filled.Login)
    object Search : Screen("search", "Поиск",
Icons.Filled.Search)
    object Favorites : Screen("favorites", "Избранное",
Icons.Filled.Star)
    object Recommendations : Screen("recommendations",
"Рекомендации", Icons.Filled.AutoAwesome)
    object Account : Screen("account", "Профиль",
Icons.Filled.AccountCircle)
    object Details : Screen(
        route = "details/{url}/{title}/{content}",
        title = "Результат"
    ) {
        fun createRoute(url: String, title: String, content:
String): String {
            return "details/" +

```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		57

```

        "${URLEncoder.encode(url, "UTF-8")}/" +
        "${URLEncoder.encode(title, "UTF-8")}/" +
        "${URLEncoder.encode(content, "UTF-8")}"
    }
}

object WebView : Screen("webview/{url}", "Брайзер") {
    fun createRoute(url: String) =
        "webview/${URLEncoder.encode(url, "UTF-8")}"
}

object History : Screen("history", "История",
Icons.Filled.History)
}

@Composable
fun SetupNavGraph(
    authState: AuthState,
    navController: NavHostController =
rememberNavController()
) {
    var isLoggedIn by remember { mutableStateOf(false) }
    LaunchedEffect(Unit) {
        authState.isLoggedIn.collectLatest { value ->
            isLoggedIn = value
        }
    }

    val currentBackStackEntry by
navController.currentBackStackEntryAsState()

    val currentRoute =
currentBackStackEntry?.destination?.route

    Scaffold(
        bottomBar = { BottomNavBar(navController, isLoggedIn,
currentRoute) }
    ) { paddingValues ->
        NavHost(
            navController = navController,

```

```

startDestination = if (isLoggedIn)
Screen.Search.route else Screen.Login.route,
modifier = Modifier.padding(paddingValues)
) {
composable(Screen.Login.route) {
LoginScreen(authState = authState,
onLoginSuccess = {
navController.navigate(Screen.Search.route)
{
popUpTo(Screen.Login.route) {
inclusive = true }
}
})
}
composable(Screen.Search.route) {
SearchScreen(navController = navController,
authState = authState)
}
composable(Screen.Favorites.route) {
FavoritesScreen(navController =
navController, authState = authState)
}
composable(Screen.Recommendations.route) {
RecommendationsScreen(navController =
navController, authState = authState)
}
composable(Screen.Account.route) {
AccountScreen(authState = authState, onLogout
= {
authState.logout()

navController.navigate(Screen.Login.route) {
popUpTo(0) { inclusive = true }
}
}
}

```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		59

```

        })
    }
    composable(
        Screen.Details.route,
        arguments = listOf(navArgument("url") { type
= NavType.StringType })
    ) { backStackEntry ->
        val
                                encodedUrl
                                =
backStackEntry.arguments?.getString("url") ?: ""
        val url = try { URLDecoder.decode(encodedUrl,
"UTF-8") } catch (e: Exception) { encodedUrl }
        DetailsScreen(navController = navController,
authState = authState)
    }
    composable(
        route = Screen.WebView.route,
        arguments = listOf(navArgument("url") { type
= NavType.StringType })
    ) { backStackEntry ->
        val
                                encodedUrl
                                =
backStackEntry.arguments?.getString("url") ?: ""
        val url = try { URLDecoder.decode(encodedUrl,
"UTF-8") } catch (e: Exception) { encodedUrl }
        WebViewScreen(url = url, navController =
navController)
    }
    composable(Screen.History.route) {
        HistoryScreen(navController = navController,
authState = authState)
    }
}

}

@Composable
fun BottomNavBar(

```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		60

```

        navController: NavHostController,
        isLoggedIn: Boolean,
        currentRoute: String?
    ) {
        val items = if (isLoggedIn) {
            listOf(Screen.Search, Screen.History,
Screen.Favorites, Screen.Recommendations, Screen.Account)
        } else {
            listOf(Screen.Search, Screen.Login)
        }
        NavigationBar {
            items.forEach { screen ->
                val selected = when {
                    currentRoute?.startsWith("details") == true -
> screen == Screen.Search
                    currentRoute?.startsWith("webview") == true -
> screen == Screen.Search
                    else -> currentRoute == screen.route
                }
                NavigationBarItem(
                    icon = {
                        screen.icon?.let {
                            Icon(
                                imageVector = it,
                                contentDescription =
screen.title,
                                modifier = Modifier.size(20.dp)
                            )
                        }
                    },
                    label = {
                        Text(
                            text = screen.title,
                            fontSize = 10.sp,
                            maxLines = 1,

```

```

        overflow = TextOverflow.Ellipsis
    )
},
selected = selected,
onClick = {
    navController.navigate(screen.route) {
        popUpTo(navController.graph.startDestinationId) { saveState =
true }

        launchSingleTop = true
        restoreState = true
    }
},
alwaysShowLabel = true
)
}
}
}
}

```

Листинг А.5 – Класс « SerpStackApi.kt»

```

package nadinee.studentmaterialssearch.network
import nadinee.studentmaterialssearch.data.SerpStackResponse
import retrofit2.http.GET
import retrofit2.http.Query
interface SerpStackApi {
    @GET("search")
    suspend fun search(
        @Query("access_key") apiKey: String,
        @Query("query") query: String,
        @Query("num") num: Int = 10,
        @Query("country") country: String = "ru",
        @Query("language") language: String = "ru",
        @Query("engine") engine: String? = null,
        @Query("device") device: String? = null
    ): SerpStackResponse
}

```

Листинг А.6 – Класс « SerpStackClient.kt»

```
package nadinee.studentmaterialssearch.network
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import nadinee.studentmaterialssearch.data.OrganicResult
import nadinee.studentmaterialssearch.data.SerpStackResponse
import nadinee.studentmaterialssearch.data.SearchResult
object SerpStackClient {
    private const val API_KEY =
"8f01dfa59291f7df5626368121a0f8c3"
    private const val BASE_URL = "https://api.serpstack.com/"
    private val retrofit = Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addConverterFactory(GsonConverterFactory.create())
        .build()
    private val api: SerpStackApi =
retrofit.create(SerpStackApi::class.java)
    suspend fun search(query: String, engine: String? =
"google"): List<SearchResult> {
        val response = api.search(API_KEY, query, engine =
engine)
        response.error?.let { error ->
            throw Exception("Serpstack: ${error.message} ?:
"Unknown error}")
        }
        val results = response.organic_results
        if (results.isNullOrEmpty()) {
            throw Exception("Нет результатов для запроса:
$query")
        }
        return results.map { result ->
            SearchResult(
                title = result.title ?: "Без заголовка",
                url = result.url ?: "",
                content = result.snippet ?: "Без описания"
```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		63

```

        )
    }
}

```

Листинг А.7 – Класс « RecommendationsScreen.kt»

```

package nadinee.studentmaterialssearch.screens

import AuthState
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Search
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.asSharedFlow
import kotlinx.coroutines.flow.collectLatest
import nadinee.studentmaterialssearch.App
import nadinee.studentmaterialssearch.data.Favorite
import nadinee.studentmaterialssearch.navigation.Screen
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun RecommendationsScreen(
    navController: NavController,
    authState: AuthState
) {
    val currentUser = authState.currentUser.collectAsState().value
}

```



```

        val favorites by produceState(initialValue =
emptyList<Favorite>()) {
            currentUser?.email?.let { email ->
                value =
App.database.favoriteDao().getAllForUser(email)
            }
        }
        val interests = currentUser?.interests
            ?.split(",")
            ?.map { it.trim() }
            ?.filter { it.isNotEmpty() } ?: emptyList()

        val favoriteWords = favorites
            .flatMap { listOf(it.title, it.content) }
            .flatMap { it.split(" ", ",", ".", "!", "?", "-",
"_" ) }
            .map { it.lowercase() }
            .filter { it.length > 4 && it.none { char -> char in
"0123456789" } } // убираем короткие и с цифрами
            .groupBy { it }
            .eachCount()
            .entries
            .sortedByDescending { it.value }
            .take(20)
            .map { it.key }
        val recommendedQueries = (interests + favoriteWords)
            .filter { it.isNotEmpty() }
            .distinct()
            .shuffled()
            .take(12)

        Scaffold(
            topBar = {
                CenterAlignedTopAppBar(
                    title = { Text("Рекомендации для вас") },

```

					КР.820000.000	Лист
Изм.	Лист	№ докум.	Подпись	Дата		65

```

        colors
TopAppBarDefaults.centerAlignedTopAppBarColors (
        containerColor
MaterialTheme.colorScheme.primaryContainer
    )
    )
}
) { padding ->
    Box(modifier = Modifier.padding(padding)) {
        if (recommendedQueries.isEmpty()) {
            Box(
                modifier = Modifier.fillMaxSize(),
                contentAlignment = Alignment.Center
            ) {
                Column(horizontalAlignment
Alignment.CenterHorizontally) {
                    Text("Добавьте интересы в профиле",
fontSize = 18.sp)
                    Spacer(Modifier.height(8.dp))
                    Text("и сохраните материалы в
избранное —", color = MaterialTheme.colorScheme.outline)
                    Text("и я подберу лучшие темы!",
color = MaterialTheme.colorScheme.outline)
                }
            }
        } else {
            LazyColumn(
                contentPadding = PaddingValues(16.dp),
                verticalArrangement
Arrangement.spacedBy(12.dp)
            ) {
                items(recommendedQueries) { query ->
                    Card(
                        onClick = {

```

```

SearchQueryEvent.sendQuery(query)

navController.navigate(Screen.Search.route) {

popUpTo(navController.graph.startDestinationId) { saveState = true
}

                                launchSingleTop = true
                                restoreState = true
                                }
                                },
                                modifier
                                =
Modifier.fillMaxWidth(),
                                colors = CardDefaults.cardColors(
                                containerColor
                                =
MaterialTheme.colorScheme.secondaryContainer
                                )
                                ) {
                                Row(
                                modifier = Modifier
                                .fillMaxWidth()
                                .padding(16.dp),
                                verticalAlignment
                                =
Alignment.CenterVertically
                                ) {
                                Icon(
                                imageVector
                                =
Icons.Default.Search,
                                contentDescription
                                =
null,
                                tint
                                =
MaterialTheme.colorScheme.primary
                                )
                                Spacer(Modifier.width(12.dp))
                                Text(

```

					КР.820000.000	Лист
						67
Изм.	Лист	№ докум.	Подпись	Дата		

